Métriques d'affinité

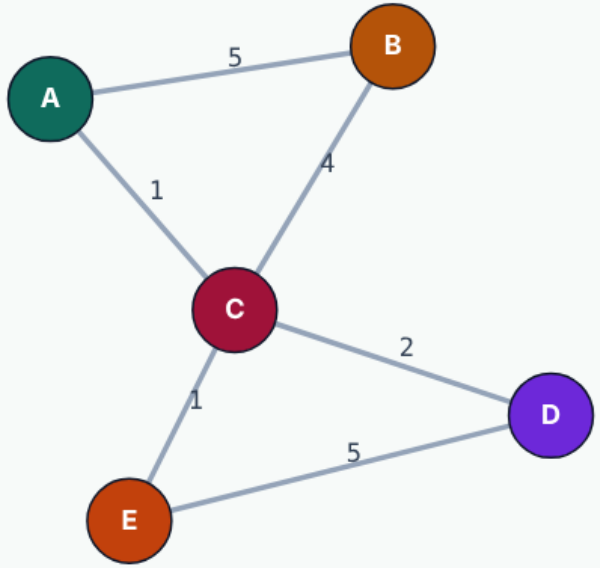
Avant de fusionner des cellules ou de réduire une netlist, il faut savoir lesquelles vont ensemble

Un petit graphe pondéré

MÉTRIQUES D'AFFINITÉ

Un petit graphe pondéré

Étape 1 / 5 · raw-graph · module01-01-affinity-metrics



Explication

Cinq cellules A–E avec des poids d'arête. Les arêtes lourdes A–B et D–E pèsent toutes les deux 5 ; le pont qui passe par C est plus faible. L'affinité va décider quelles paires veulent former un cluster en premier.

- Nœuds = cellules ou clusters à regrouper
- Poids d'arête = à quel point une paire doit rester unie
- Instance de référence utilisée dans tous les labs de clustering

```
Graphe : 5 nœuds, 6 arêtes
Arêtes les plus lourdes : A–B=5, D–E=5
```

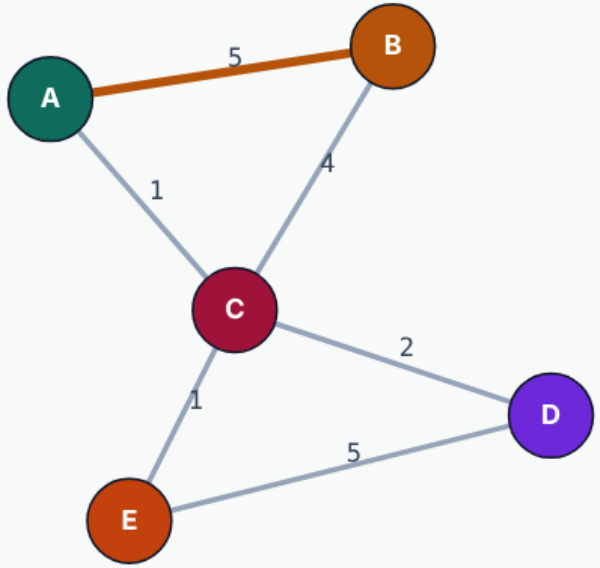
Un petit graphe pondéré

L'affinité par poids d'arête classe les paires

MÉTRIQUES D'AFFINITÉ

L'affinité par poids d'arête classe les paires

Étape 2 / 5 · edge-rank · module01-01-affinity-metrics



Explication

L'affinité par poids d'arête trie simplement les arêtes existantes. A–B et D–E sont à égalité à 5 ; le départage place A–B en premier. Ce classement est la priorité de fusion la plus simple.

- score(u,v) = w(u,v)
- Tri par score décroissant
- La première paire devient la première candidate à la fusion

```
1. A–B @ 5
2. D–E @ 5
3. B–C @ 4
4. C–D @ 2
5. A–C @ 1
6. C–E @ 1
```

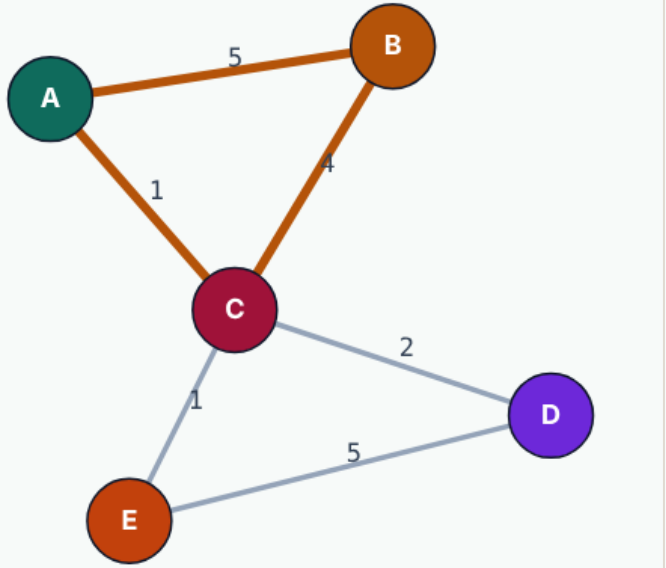
L'affinité par poids d'arête classe les paires

Les voisins communs renforcent A–B

MÉTRIQUES D'AFFINITÉ

Les voisins communs renforcent A–B

Étape 3 / 5 · shared-idea · module01-01-affinity-metrics



Explication

A et B partagent déjà une arête de poids 5. Ils ont aussi un voisin commun, C. L'affinité par voisins communs ajoute min(w(A,C), w(B,C)) = min(1,4) = 1, donc A–B monte à 6.

- L'arête directe compte toujours
- Plus le soutien des voisins communs
- Favorise le regroupement des triangles denses

```
w(A,B)=5
via C : min(1,4)=1
score(A,B)=6
```

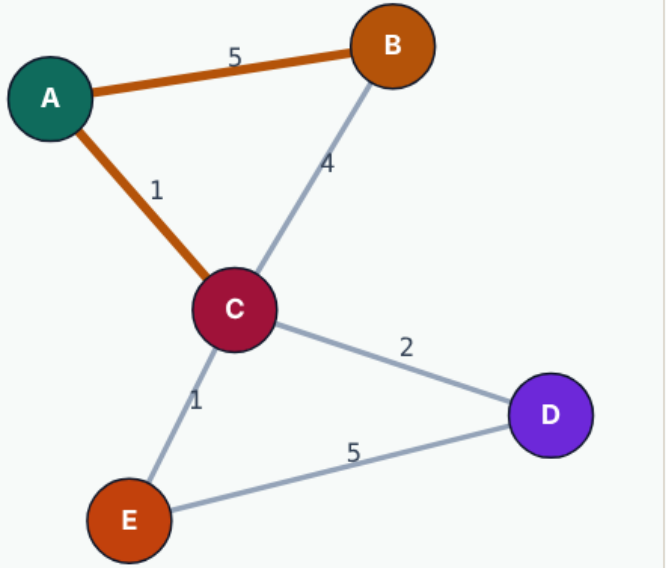
Les voisins communs renforcent A–B

Le classement par voisins communs rebat les cartes

MÉTRIQUES D'AFFINITÉ

Le classement par voisins communs rebat les cartes

Étape 4 / 5 · shared-rank · module01-01-affinity-metrics



Explication

Avec le score par voisins communs, A–B et D–E mènent à 6. L'arête faible A–C bondit à 5 grâce à B. Des paires sans arête, comme B–D, peuvent apparaître grâce au seul soutien de leurs voisins.

- A–C : 1 avant → 5 maintenant
- Paires sans arête (ex. B–D @ 2) : attraction transitive
- La même idée alimente les priorités de la fusion gloutonne

```
1. A–B @ 6
2. D–E @ 6
3. A–C @ 5
4. B–C @ 5
5. C–D @ 3
6. C–E @ 3
```

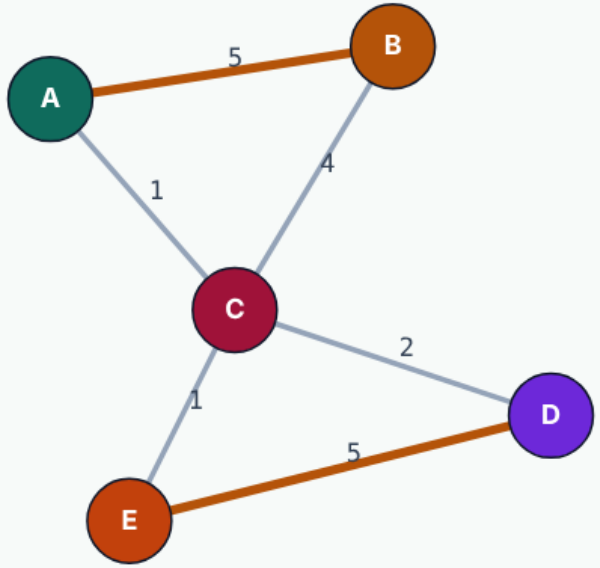
Le classement par voisins communs rebat les cartes

Pourquoi l'affinité compte en EDA

MÉTRIQUES D'AFFINITÉ

Pourquoi l'affinité compte en EDA

Étape 5 / 5 · takeaway · module01-01-affinity-metrics



Explication

L'affinité est la préférence locale de l'objectif : quelles cellules vont ensemble, avant les contraintes globales. Mauvaise affinité → mauvaises fusions → raffinement plus difficile ensuite.

- Poids d'arête = connectivité, nets partagés
- Voisins communs ≈ groupes locaux plus denses
- Les labs suivants utilisent ce classement pour fusionner

```
Meilleure paire (arête) : A–B @ 5
Meilleure paire (voisins communs) : A–B @ 6
Le graphe d'exemple reste fixe pour les
résultats de référence
```

Pourquoi l'affinité compte en EDA

Le score par voisins communs en pseudo-code

- Voici le score par voisins communs sous forme de code
- D'abord, chaque arête existante ajoute son poids à sa paire
- Ensuite, on prend chaque paire de nœuds et chacun de leurs voisins communs
- Pour chaque voisin commun, on ajoute le plus petit des deux poids vers la paire
- Une paire dont le score dépasse zéro est gardée
- Enfin, on trie par score décroissant, et on départage les égalités par ordre alphabétique

Le score par voisins communs en pseudo-code — à essayer

```
affinity_shared_neighbors(nodes, edges):
  score[u,v] = w(u,v) pour chaque arête
  pour chaque paire (u, v) de nœuds :
    s = 0
    pour x dans voisins(u), x != v :
      si x dans voisins(v) :
        s += min(w(u,x), w(v,x))
    si s > 0 ou (u,v) a une arête :
      score[u,v] += s
  trier par score (décroissant), puis u, puis v
```

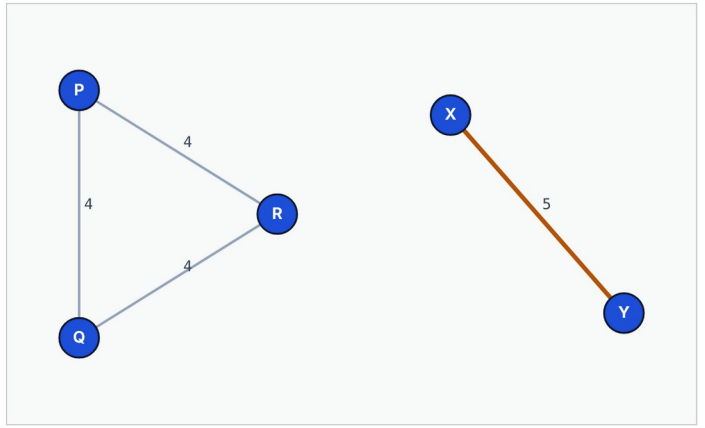
Dix paires, pas six

- Avec le score par voisins communs, le graphe de départ donne dix paires, et non six
- Quatre d'entre elles n'ont aucune arête
- B et D sont tous les deux reliés à C, donc B–D obtient le plus petit rayon : deux
- A–D, A–E et B–E gagnent chacune un, en passant par C
- Ces paires ne sont pas des erreurs
- Elles disent que deux cellules sont proches, même sans fil direct

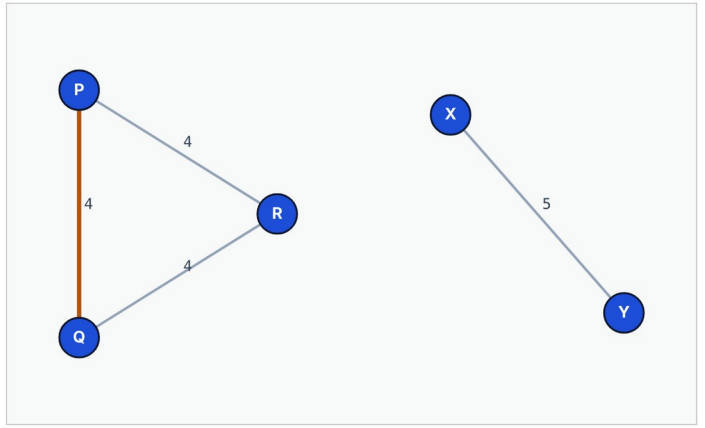
Même graphe, première fusion différente

- Prenons un triangle serré P, Q, R, dont chaque arête pèse quatre
- Ajoutons une paire isolée X–Y, de poids cinq
- Le poids d'arête choisit d'abord X–Y, car cinq bat quatre
- Le score par voisins communs donne à P–Q quatre, plus quatre via R : huit
- X–Y n'a aucun voisin commun, donc reste à cinq
- Même graphe, même moteur de fusion, mais une première fusion différente

Poids d'arête : 1re fusion X–Y (5)



Voisins communs : 1re fusion P–Q (4+4 = 8)



Des nets à nombreuses broches

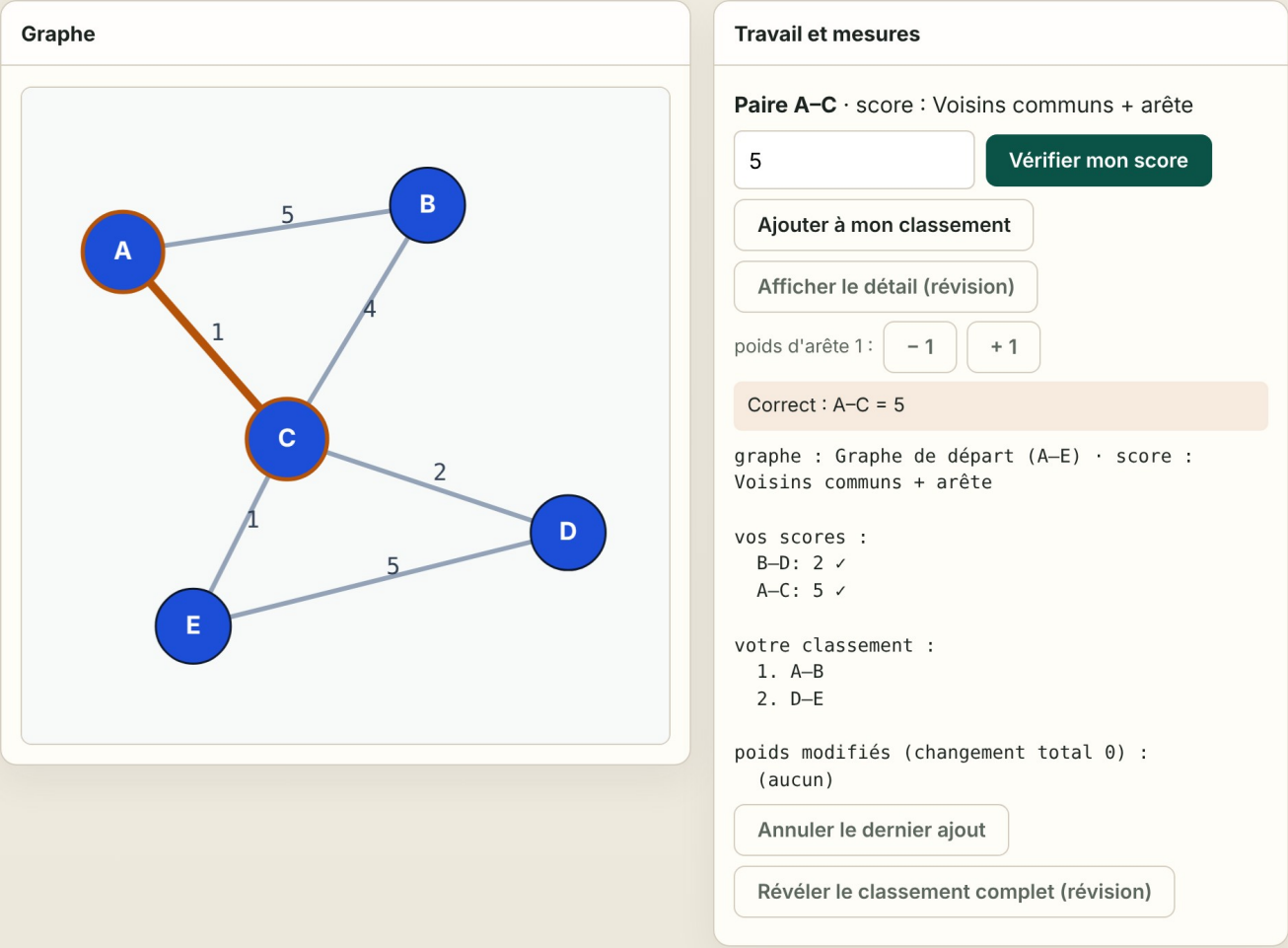
- Les vraies netlists ne sont pas faites de simples arêtes
- Un seul net peut relier de nombreuses cellules, comme une ligne de reset ou d'enable
- Compter chaque net partagé à plein poids rapproche A et B à tort
- Elles semblent proches simplement parce qu'elles voient toutes les deux le reset
- Le score par nombre de broches corrige cela
- Chaque net divise son poids par son nombre de broches moins un

Des nets à nombreuses broches — à essayer

petite netlist : en {A,B,C,D,E}, rst {A,B,C,D,E}, n1 {E,F}

	nombre brut de nets	nombre de broches w/(|e|-1)
A-B	1 + 1 = 2	1/4 + 1/4 = 0.5
E-F	1	1/1 = 1.0
1re fusion	A-B	E-F

Le parcours lab dans le navigateur



Le lab dans le navigateur

Le parcours implémentation

wsl — module01-01-affinity-metrics/examples/

```
# Track A — real Unix
$ python3 ../common/solvers.py examples/priority_graph.json --mode affinity
affinity edge-weight (desc):
  X-Y: 5.0
  P-Q: 4.0
  P-R: 4.0
  Q-R: 4.0
affinity shared-neighbors+edge (desc):
  P-Q: 8.0
  P-R: 8.0
  Q-R: 8.0
  X-Y: 5.0

$ python3 examples/affinity_check.py
checking reference
PASS  tiny graph, edge weight: 6 pairs in golden order
PASS  tiny graph, shared neighbors: 10 pairs in golden order
PASS  priority graph, edge weight: first merge X-Y @ 5
PASS  priority graph, shared neighbors: first merge P-Q @ 8
PASS  netlist, pin-count: first merge E-F @ 1, A-B only 0.5
all goldens match
```

Session dans un vrai terminal

Le parcours implémentation — à essayer

```
# indiquer à Python où sont les fonctions partagées
export PYTHONPATH=../common

# les deux classements : graphe de départ, puis graphe de priorité
python3 ../common/solvers.py examples/tiny_graph.json --mode
affinity
python3 ../common/solvers.py examples/priority_graph.json --mode
affinity

# vérifier tous les résultats de référence sur vos propres fonctions
python3 examples/affinity_check.py --impl my_affinity.py
```

Les pièges à éviter

- Attention à quatre choses
- Compter deux fois une arête non orientée, une fois depuis chaque extrémité
- Ajouter le plus grand rayon au lieu du plus petit, ce qui rend tout proche de tout
- Oublier le départage : A–B et D–E changent alors de place d'une exécution à l'autre
- Et compter les nets bruts sur une netlist
- Des nets très larges comme le reset collent alors des cellules sans rapport

À vous de jouer

- Faites passer les trois vérifications de référence
- Puis expliquez pourquoi A–C monte avec le score par voisins communs
- Expliquez aussi pourquoi le graphe de priorité change sa première fusion
- Faites le petit quiz
- Passez ensuite à la fusion gloutonne par paires, où ces scores deviennent la règle